Q1 Classify operating systems and explain different OS structures (Monolithic, Layered, Microkernel, Virtual Machine) with neat diagrams. Explain multiprogramming, batch processing, time-sharing, real-time systems. What is the need for protection and how is it achieved?

Introduction: An Operating System is system software that manages hardware and software resources, acting as an intermediary between users and hardware. Understanding OS types and internal structures is fundamental to computer science.

Types of Operating Systems:

Batch Processing: Jobs with similar requirements are grouped and processed sequentially without user interaction. OS automatically moves from one job to the next. Advantage: high CPU utilisation. Disadvantage: no interactivity, long turnaround time. Example: IBM mainframe payroll processing.

Multiprogramming: Multiple jobs reside in memory simultaneously. When one job waits for I/O, CPU switches to another — CPU is never idle. Significantly increases throughput.

Time-Sharing (Multitasking): Extension of multiprogramming where CPU switches between jobs so rapidly (time quantum) that each user feels they have a dedicated machine. Enables interactive use. Response time typically under 1 second. Example: UNIX, Linux.

Real-Time OS: Processes data within strict time constraints. Hard RTOS: missing deadline = catastrophic failure (aircraft control, pacemakers). Soft RTOS: occasional miss tolerable (multimedia streaming). Example: VxWorks, FreeRTOS.

Distributed OS: Manages networked independent computers as one coherent system. Users access remote resources transparently. Provides fault tolerance and load balancing.

Embedded OS: Designed for resource-constrained devices. Example: Android in smartphones, FreeRTOS in microcontrollers.

OS Structures:

Monolithic Structure: All OS services (file system, device drivers, memory management) packed into one large kernel. No module separation. Advantage: fast — direct function calls. Disadvantage: extremely difficult to maintain; one bug crashes entire system. Example: original UNIX.

Layered Structure: OS divided into layers (0=hardware, N=user interface). Each layer uses only services from the layer directly below. Advantage: modularity, easy debugging. Disadvantage: performance overhead; difficult to strictly define layer boundaries. Example: THE system.

Microkernel Structure: Minimal kernel contains only IPC, basic memory management, and elementary scheduling. All other services (file system, drivers, networking) run as user-space server processes communicating via message passing. Advantage: reliable (server crash doesn't bring down kernel), extensible, portable, more secure. Disadvantage: performance overhead from user-kernel context switches and message passing. Example: Mach (macOS foundation), QNX, MINIX.

Virtual Machine: Hypervisor layer sits between hardware and multiple OS instances, giving each the illusion of dedicated hardware. Type 1 (bare-metal): runs directly on hardware — VMware ESXi, Xen. Type 2 (hosted): runs on host OS — VirtualBox. Advantage: complete isolation between VMs, run multiple OS simultaneously.

Q2 Explain the purpose of system calls with categories and examples (process control, file management, device management, communication). Describe services provided by an OS. Explain storage structure and hierarchy (registers → cache → RAM → disk). Write a note on I/O structure and DMA.

System Calls, OS Services, Storage Hierarchy & DMA

Introduction: An OS provides a rich set of services to users and programs. System calls are the programmatic interface through which user programs request these services from the kernel. The storage hierarchy determines how data moves between CPU and storage. Understanding these mechanisms is central to OS design and performance.

Services Provided by an OS:

User-facing services: Program execution (load and run programs, report errors), I/O operations (users cannot directly control hardware), file system manipulation (create/read/write/delete files), communication between processes (shared memory or message passing), error detection (hardware faults, illegal memory access, arithmetic overflow).

System-efficiency services: Resource allocation (CPU, memory, I/O devices among competing processes), accounting (track usage per user/process for billing and auditing), protection and security (prevent unauthorised access to resources).

System Calls: A system call is the mechanism by which a user-space program requests a kernel service. When called, a trap instruction switches CPU to kernel mode, control transfers to the OS, which performs the service and returns the result. System calls are usually accessed through library APIs (POSIX API in Linux, Win32 in Windows) rather than directly. Example: printf() in C internally calls the write() system call.

Storage Structure and Hierarchy: Computer storage organised by speed, cost, and volatility:

Registers: Inside CPU. Sub-nanosecond access. Hundreds of bytes. Fastest, most expensive.

L1/L2/L3 Cache: On/near CPU chip. 1–40 ns. 32KB–64MB. Caches frequently used data using locality of reference principle.

Main Memory (RAM): ~100 ns. GBs. Volatile — lost on power off. Programs must be in RAM to execute.

SSD: ~0.1 ms. Hundreds of GB. Non-volatile, fast, no moving parts.

HDD: ~10 ms. TBs. Non-volatile. Cheapest per GB. Mechanical — slowest.

Optical/Tape: Slowest, cheapest. Used for long-term archival backup.

The principle of locality (temporal: recently used data likely used again; spatial: nearby data likely accessed) makes caching highly effective.

I/O Structure:

Programmed I/O (Polling): CPU continuously checks device status register to see if operation complete. Wastes CPU cycles — CPU spins doing nothing. Suitable only for very fast, brief operations.

Interrupt-Driven I/O: CPU initiates I/O and continues executing other processes. When device completes, it sends a hardware interrupt. CPU suspends current execution, saves state, executes the Interrupt Service Routine (ISR), then resumes. ISR is located through the interrupt vector table (array of ISR addresses indexed by interrupt number). Much more efficient than polling.

DMA (Direct Memory Access): For large bulk transfers (disk reads, network), the DMA controller handles data movement directly between device and main memory without CPU involvement. CPU programs DMA controller with: source address, destination address in RAM, byte count, and direction. DMA autonomously transfers data, cycle-stealing bus access when CPU doesn't need it. On completion, DMA sends ONE interrupt to CPU. CPU is completely free during the entire transfer. Example: Reading a 10MB file — DMA moves all 10MB from disk buffer to RAM while CPU executes other processes; only one interrupt at the end.

Q5 Define deadlock. State and explain the four necessary conditions (Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait). Differentiate deadlock prevention vs avoidance vs detection. Explain Banker's Algorithm for deadlock avoidance with a complete numerical example. How does a system recover from deadlock?

Deadlock: Conditions, Prevention, Avoidance & Banker's Algorithm

Introduction: Deadlock is a situation where a set of processes are permanently blocked because each holds a resource and waits for another resource held by another process in the set — a circular dependency from which no process can proceed. It is a critical OS problem that can halt an entire system.

Four Necessary Conditions (Coffman Conditions): All four must hold simultaneously for deadlock:

1. *Mutual Exclusion:* At least one resource held in non-shareable mode — only one process can use it at a time. Others must wait.
2. *Hold and Wait:* A process holds at least one resource while waiting for additional resources held by other processes.
3. *No Preemption:* Resources cannot be forcibly taken from a process. Released only voluntarily after the process finishes using them.
4. *Circular Wait:* A set of processes {P0, P1, ..., Pn} exists such that P0 waits for P1's resource, P1 waits for P2's, ..., Pn waits for P0's — forming a cycle.

Deadlock Prevention: Eliminate at least one of the four conditions:

Eliminate Hold and Wait: Require processes to request all resources at once before starting. If any unavailable, request nothing. Disadvantage: low resource utilisation, possible starvation.

Allow Preemption: If a process holding resources requests an unavailable resource, preempt all its held resources. Only works for resources whose state can be saved/restored (CPU, memory).

Eliminate Circular Wait: Impose total ordering on all resource types. Processes must always request resources in increasing order. Example: number resources 1–N; always request in ascending order.

Deadlock Avoidance — Banker's Algorithm: Requires each process to declare maximum resource needs in advance. OS dynamically evaluates each request to ensure system stays in a safe state.

Safe State: A state is safe if there exists a safe sequence — an ordering of all processes such that each process's remaining needs can be satisfied using currently available resources plus resources held by all preceding processes in the sequence. Safe state → no deadlock. Unsafe state → possible deadlock.

Data structures (n processes, m resource types):

- Available[m]: currently available instances of each resource type.
- Max[n][m]: maximum demand of each process.
- Allocation[n][m]: currently allocated to each process.
- Need[n][m] = Max − Allocation: remaining needs.

Safety Algorithm:

1. Work = Available; Finish[i] = false for all i.
2. Find i where Finish[i]=false AND Need[i] ≤ Work.
3. If found: Work += Allocation[i], Finish[i] = true. Repeat step 2.
4. If all Finish[i]=true → safe state.

Resource Request Algorithm: When Pi requests Request[i]:

1. If Request[i] > Need[i] → error.
2. If Request[i] > Available → wait.
3. Tentatively: Available−=Request[i], Allocation[i]+=Request[i], Need[i]−=Request[i].
4. Run safety algorithm. If safe → grant. If unsafe → rollback and wait.

Q6 Explain paging for memory management — logical vs physical address, page table, TLB. Differentiate internal vs external fragmentation. Explain segmentation and combined paged segmentation.

Paging, Page Replacement & Segmentation

Introduction: Memory management is a critical OS function that must allocate memory to multiple processes, provide protection, and give each process the illusion of a large address space. Paging is the dominant mechanism, extended by virtual memory and page replacement algorithms when physical memory is insufficient.

Fragmentation: *Internal Fragmentation:* Wasted space inside an allocated memory block. Example: process needs 5.5KB, gets 8KB (2 pages of 4KB) — 2.5KB wasted inside the second page. *External Fragmentation:* Total free memory is sufficient but scattered in non-contiguous small holes that cannot satisfy a large contiguous request. Paging eliminates external fragmentation.

Paging: Physical memory divided into fixed-size frames. Logical memory divided into pages of the same size (typically 4KB). A page table per process maps page numbers to frame numbers.

Address Translation: Logical address split into page number (p) and page offset (d). Page table gives frame number (f). Physical address = f × page_size + d.

TLB (Translation Lookaside Buffer): Hardware cache for recent page→frame mappings. ~98% hit rate → nearly instant translation. On miss, access page table in RAM. Effective Access Time = hitrate × TLB_time + (1−hitrate) × (TLB_time + memory_time).

Virtual Memory and Demand Paging: Allows a process to run even when not all its pages are in memory. Pages loaded on demand. On accessing a page not in memory → page fault. OS handles: save state → find free frame → load page from disk → update page table → restart instruction.

Page Replacement Algorithms: When all frames are full and a new page is needed, select a victim page to evict.

FIFO: Replace the oldest page (longest in memory). Simple — use a queue. Belady's Anomaly: more frames may cause more faults.

Optimal (OPT): Replace page not used for longest time in future. Minimum possible faults — optimal benchmark. Not implementable in practice (needs future knowledge).

LRU (Least Recently Used): Replace page not used for longest time in past. Good approximation of Optimal. Implementation: maintain timestamps or use a stack (most recent at top, LRU at bottom).

Segmentation: Segmentation divides a process into logical segments of variable size (main program, function, stack, heap, symbol table). Logical address = (segment number, offset). Segment table: each entry has base (physical start address) and limit (segment length). Access protection: if offset ≥ limit → segmentation fault.

Advantage: matches program's logical structure — programmer-friendly. Disadvantage: variable-size segments cause external fragmentation (same problem as variable-partition allocation).

Paged Segmentation (Combined): Each segment is divided into fixed-size pages. Eliminates external fragmentation of pure segmentation while preserving logical segment structure. Address translation: segment number → segment table → page table for that segment → frame number + offset. Used by Intel x86 architecture. Advantage: gets best of both — logical structure of segments, efficiency of paging.

Q7 Explain disk scheduling algorithms: FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK. Apply them on a given request sequence with head starting at a given track. Calculate total head movement and determine which algorithm is most efficient. Write a note on DMA and spooling.

Disk Scheduling Algorithms with Calculations

Introduction: Hard disk drives store data on rotating magnetic platters in concentric tracks divided into sectors. The disk head must physically move (seek) to the correct track before reading or writing. Seek time (5–15ms) dominates disk access time. Since many processes have pending disk I/O, the OS schedules disk requests to minimise total head movement, maximising throughput and minimising response time.

Disk Access Time = Seek Time + Rotational Latency + Transfer Time. Seek time is dominant and directly controllable by the OS scheduling algorithm.

Given Example (2023 paper): 200 tracks (0–199). Requests: {82, 170, 43, 140, 24, 16, 190}. Head starts at track 50.

1. FCFS (First Come First Served): Serve requests in arrival order regardless of head position.

tracks

Simple and fair but highly inefficient — ignores spatial locality of requests.

2. SSTF (Shortest Seek Time First): Always service the request closest to current head position. Greedy approach. Much more efficient than FCFS. Problem: Starvation — requests at extremes may wait indefinitely as closer requests keep arriving.

3. SCAN (Elevator Algorithm): Head moves in one direction servicing all requests, reaches the last request in that direction, then reverses. Like an elevator.

Eliminates starvation. Maximum wait time = time for two full sweeps of disk.

4. C-SCAN (Circular SCAN): Like SCAN but after reaching one end, head jumps back to the other end without servicing requests on return. Provides more uniform wait time.

5. LOOK and C-LOOK: LOOK: Like SCAN but head only moves as far as the last request in each direction — doesn't travel to physical disk end. More efficient than SCAN.

C-LOOK: Like C-SCAN but jumps from last request at high end directly to first request at low end (not to track 0). More efficient than C-SCAN.

Comparison Table:

Algorithm	Total Movement	Notes
FCFS	642 tracks	Simple, inefficient
SSTF	208 tracks	Best movement, starvation risk
SCAN	314 tracks	Fair, no starvation
C-SCAN	373 tracks	Most uniform wait times

DMA (Direct Memory Access): DMA allows I/O devices to transfer data directly to/from main memory without CPU involvement. CPU programs DMA controller with: source (device buffer), destination (RAM address), byte count, and direction. DMA controller autonomously transfers data, cycle-stealing bus access when CPU doesn't need it. On completion, DMA sends ONE interrupt to CPU. CPU is completely free during entire transfer. Example: Reading 1MB file from disk — DMA transfers all 1MB to RAM while CPU runs other processes; only one interrupt at the end. Dramatically reduces CPU overhead for bulk I/O.

Spooling (Simultaneous Peripheral Operations On-Line): Output destined for a slow device (printer) is first written to a fast disk buffer (the spool). OS daemon reads from spool and feeds the slow device at its pace. Applications don't block waiting for the slow device — they write to spool and continue immediately. Example: Printing — document data goes to print spool disk buffer instantly, application returns to user control, print spooler daemon feeds data to printer in the background.

Q8 Discuss file allocation methods in detail: Contiguous, Linked, and Indexed allocation — with advantages and disadvantages of each. Explain directory structures: single-level, two-level, tree-structured. Explain file system protection, access control lists, and the UNIX file system.

File Allocation Methods, Directory Structures & UNIX File System

Introduction: A file system is the OS component responsible for organising, storing, retrieving, protecting, and naming files on persistent storage. Two fundamental design questions are: how to allocate disk space to files, and how to organise the file namespace. This answer covers all three allocation methods, directory structures, and the UNIX file system.

File Allocation Methods:

1. Contiguous Allocation: Each file occupies a set of adjacent disk blocks. Directory entry stores only starting block number and file length. Example: file starting at block 14 with length 4 occupies blocks 14, 15, 16, 17.

Advantages: Simple — only start and length needed. Excellent read performance — sequential or random access both fast. Supports direct access: to read block i, access block (start+i).

Disadvantages: External fragmentation — holes scattered across disk over time. File cannot grow without moving elsewhere. Must declare file size at creation time.

2. Linked Allocation: Each file is a linked list of disk blocks anywhere on disk. Each block contains a pointer to the next block. Directory stores first and last block addresses.

Example: file starts at block 9 → 16 → 1 → 10 → 25 → NULL.

Advantages: No external fragmentation — any free block usable. Files grow dynamically — just allocate any free block.

Disadvantages: Sequential access only — random access requires following pointers from start O(n). Pointer overhead wastes space inside blocks. Poor reliability — corrupt pointer loses all subsequent data.

FAT (File Allocation Table): Brings all pointers into a table in memory. FAT[i] = next block number (or EOF/free). Faster traversal since FAT cached in RAM. Used by USB drives, SD cards.

3. Indexed Allocation: Each file has a dedicated index block containing an array of pointers to all its data blocks. Directory stores only the index block address.

Advantages: Direct access O(1) — read index_block[i] to get block i. No external fragmentation. No pointer overhead in data blocks.

Disadvantages: Index block overhead — even tiny files need a full index block. Single index block insufficient for very large files.

Multi-level Indexing (UNIX inode): 12 direct pointers + 1 single indirect (points to block of pointers) + 1 double indirect + 1 triple indirect. Supports files from tiny to hundreds of GB.

Comparison:

Method	External Fragmentation	Direct Access	Grow File	Reliability
Contiguous	Yes	Excellent	Difficult	Good
Linked	No	Poor O(n)	Easy	Poor
Indexed	No	Good O(1)	Easy	Good